

PROBLEM STATEMENT 1: AI-ASSISTED BUG DETECTION**ORIGINAL CODE:**

```
def factorial(n):    result = 1    for i in range(1, n):        result = result * i    return result
```

TEST RUN:

```
>>> factorial(5)
24
```

AI PROMPT USED: "IDENTIFY THE LOGICAL BUG IN THIS FACTORIAL FUNCTION AND PROVIDE A CORRECTED VERSION." AI RESPONSE SUMMARY: OFF-BY-ONE ERROR. RANGE(1, N) GOES TO N-1, SO MISSES N.

CORRECTED:

```
def factorial(n):    if n < 0:        raise ValueError("Factorial not defined for negative numbers")    result = 1    for i in range(1, n+1):        result *= i    return result
```

TEST CORRECTED:

```
>>> factorial(5)
120 >>> factorial(0)
1
>>> factorial(-1)
ValueError: Factorial not defined for negative numbers
```

COMPARISON: AI CAUGHT EDGE CASES LIKE NEGATIVE AND ZERO; MY MANUAL FIX MISSED ZERO INITIALLY.

PROBLEM STATEMENT 2: IMPROVING READABILITY & DOCUMENTATION

ORIGINAL CODE:

```
def calc(a, b, c):    if c == "add":        return a + b    elif c == "sub":
return a - b    elif c == "mul":        return a * b
    elif c == "div":
```

AI PROMPT: "CRITIQUE READABILITY AND REWRITE WITH DOCSTRING, VALIDATION, ERROR HANDLING."

AI IMPROVED VERSION:

```
def calculate(a: float, b: float, operation: str) -> float: """ Perform basic arithmetic
operations on two numbers.
```

Args:

a (float): First number. b (float): Second

number.

operation (str): Operation to perform ('add', 'sub', 'mul', 'div').

Returns:

float: Result of the operation. Raises:

ValueError: If invalid operation or division by zero.

TypeError: If a or b not numbers.

Examples:

```
>>> calculate(10, 5, 'add')
```

```
15.0
```

```
""" if not isinstance(a, (int, float)) or not isinstance(b, (int, float)):
```

```
    raise TypeError("Inputs must be numbers")
```

```
operations = {      'add': a + b,
```

```
    'sub': a - b,
```

```
    'mul': a * b,      'div': a / b if b != 0 else _raise_div_zero()
```

```
}      if operation not in operations:
```

```
    raise ValueError("Invalid operation")
```

```
return operations[operation] def
```

```
_raise_div_zero():
```

```
    raise ValueError("Division by zero")
```

Tests:

```
>>> calculate(10, 5, 'add')
15.0
>>> calculate(10, 0, 'div')
ValueError: Division by zero
>>> calculate('a', 5, 'add') TypeError: Inputs must be numbers
```

PROBLEM STATEMENT 3: ENFORCING CODING STANDARDS

ORIGINAL:

```
def Checkprime(n):
    for i in range(2, n):
        if n % i == 0:
            return False
    return True
```

PEP8 COMPLIANT:

```
def is_prime(n): """Check if n is
prime."""
    if n <= 1:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True
```

Tests:

```
>>> is_prime(17)
True
>>> is_prime(4)
False
>>> is_prime(1)
False
```

PROBLEM STATEMENT 4: AI AS CODE REVIEWER

ORIGINAL:

```
def processData(d):  
    return [x * 2 for x in d if x % 2 == 0]
```

MANUAL REVIEW: POOR NAMING (PROCESSDATA VAGUE), NO TYPES/VALIDATION, ASSUMES

LIST/INTS, NO DOC. AI REVIEW PROMPT: "REVIEW FOR READABILITY, ADD VALIDATION/TYPE

HINTS, GENERALIZE."

IMPROVED:

```
from typing import List, Union, Callable  
def double_even_numbers(numbers: List[Union[int, float]]) -> List[Union[int, float]]:  
    """ Double even numbers in a list.  
    Args:  
        numbers: List of numbers. Returns:  
        List of doubled even numbers.  
    Raises:  
        TypeError: If not list. """  
    if not isinstance(numbers, list):  
        raise TypeError("Input must be a list")  
    return [num * 2 for num in  
            numbers  
            if isinstance(num, (int, float)) and num % 2 == 0]
```

Tests:

```
>>> double_even_numbers([1, 2, 3, 4])  
[4, 8]  
>>> double_even_numbers("not list")  
TypeError: Input must be a list
```

PROBLEM STATEMENT 5: PERFORMANCE OPTIMIZATION

ORIGINAL:

```
def sum_of_squares(numbers):  
    total = 0    for num in numbers:  
total += num ** 2    return total
```

TEST LARGE INPUT:

```
import time  
nums = list(range(1000000))  
start = time.time()  
sum_of_squares(nums)  
print(time.time() - start) # 0.145s (approx)
```

AI ANALYSIS: $O(N)$ FINE, BUT LOOP SLOWER THAN SUM(GENEXP).

OPTIMIZED:

```
def sum_of_squares(numbers):  
    return sum(x ** 2 for x in numbers)
```

TEST:

```
start = time.time()  
sum_of_squares(nums)  
print(time.time() - start) # 0.035s  
(approx)
```

FASTER BY 4X USING BUILT-IN SUM. TRADE-OFF: SLIGHTLY LESS READABLE FOR BEGINNERS, BUT STANDARD IDIOM.

EXECUTION TIMES COMPARISON

Function	Time (s) for 1M nums
Original loop	0.145
Optimized sum	0.035